

خطة مشروع: منصة إدارة مخزون (SaaS) ومبيعات الألبسة

1. فكرة المشروع

لإدارة مخزون ومبيعات محلات (internal) داخلية SaaS منصة
مستقل تماماً (account) كل محل **multi-tenant** الألبسة، بنظام
account. داخل كل (roles) عن الباقي، مع نظام صلاحيات

2. البنية الهرمية (Hierarchy)

Platform

└─ Super Admin - يدير كل الـ accounts على
المنصة (تفعيل/تعطيل، مراقبة عامة)

└─ Account (كل محل) - منفصل تماماً عن باقي الحسابات

└─ Owner - كل الصلاحيات

└─ Warehouse Employee (موظف)

صلاحيات محدودة - (account واحد بس لكل

Role صلاحيات كل

القسم	Super Admin	Owner	Warehouse
إدارة كل ال accounts	✓	✗	✗
Products (إضافة/تعديل)	-	✓	✓
Homepage / Categories	-	✓	✓
Business Part (تسعير، مصاريف)	-	✓	✗
Accounting Part	-	✓	✗
سعر Cost رؤية الجملة	-	✓	✗
إضافة موظف Warehouse	-	✓	✗

3. ال Product Template

كل منتج يبحوي

- **Category** (Pantalon, Chemise... إلخ)
- صورة المنتج

- اللون (Color)
- فقط يقدر يشوفه/يدخله Owner الـ (Cost) سعر الجملة
- الكمية المتوفرة (Stock)
- محسوب أوتوماتيك، قابل (Suggested Price) سعر البيع للتعديل يدوياً وقت البيع

منطق إضافة منتج جديد

1. Warehouse (Category, Image, Color, Stock) بضيف منتج (السعر بيضل فاضي مؤقتاً) Cost بدون
2. Owner عندها يتحسب Cost → بيرجع لاحقاً ويضيف الـ suggestedPrice أوتوماتيك
3. Warehouse جديدة categories كمان بيقدر يضيف

Restocking (كمية إضافية)

،(category + color نفس) لو وصلت شحنة جديدة لنفس المنتج بس،(جديد entry مش) بتزيد على الكمية الموجودة لنفس المنتج لتتبع تاريخ كل (StockMovement) بتنسجل بجدول منفصل شحنة وكلفتها (ممكن تختلف كلفة شحنة عن الثانية)

4. Business Part (قسم التسعير والتحليل)

فقط. بيحتوي Owner خاص بالـ

1. إدخال المصاريف العامة (إيجار، كهرباء، رواتب...) - بتنضاف يدوياً كل ما صار مصروف فعلي (مش تسجيل شهري ثابت)
2. حساب السعر الأوتوماتيكي لكل قطعة

نصيب القطعة من المصاريف = مجموع المصاريف بالفترة ÷ عدد القطع (مباعة أو موجودة) بنفس الفترة

السعر المقترح = (سعر الجملة + نصيب القطعة من المصاريف) × (category + 1 + نسبة الربح الخاصة بال)

3. مش - (يومي/أسبوعي job) الحساب بيصير بشكل دوري لحظي بكل مرة، حتى يكون دقيق وما يبطي النظام
4. السعر الناتج هو **سعر مقترح** - البائع فيه يعدله يدوياً وقت البيع لهيك عملية بس، من `Sale.soldPrice` بيتسجل بـ) الفعلي (دون ما يغير السعر الافتراضي للمنتج
5. تحليل أداء عام عبر فترات - **(Analytics) دراسة بيانات** زمنية

5. الجردة (Reports)

- يومية وأسبوعية
- بتحتوي: المخزون + المبيعات + الأسعار (كل شي سوا)
- كل live مش) **scheduled aggregation job** تتحسب كـ حتى تطلع `DailyReport` بتتخزن جاهزة بجدول - (مرة بسرعة

6. الصور (Images)

القرار	التفصيل
حد أقصى Upload	لكل صورة 5-10 MB
Processing	أوتوماتيك وقت Resize + Compress (Sharp مكتبة) الرفع
الصيغة	WebP (بـ 35-25 JPEG أخف من)
النسخ المخزنة	Thumbnail (~150px) + Display مش الأصلية - (~800px)
مكان التخزين	Object storage (Cloudflare R2 أو S3) يلي URL بس الـ database، مش بال - بيتخزن
تقدير الحجم الكلي	منتج 100 × seller لـ 10,000 ~150 GB (رقم معقول وكلفته زهيدة)

7. التقني Stack الـ

الطبقة	التقنية
Frontend	React (Vite) + TypeScript + Tailwind
Backend	Node.js + Express/Fastify + TypeScript

الطبقة	التقنية
Database	PostgreSQL
ORM	Prisma
Auth	(Clerk أو Supabase Auth) حل جاهز بدل بناءه من الصفر
Image Storage	Cloudflare R2 أو AWS S3
Hosting	Railway / Render (backend + DB), اختياري Vercel (frontend)
Connection Pooling	PgBouncer (إما مدمج بالـ managed DB أو self-hosted بلا كلفة إضافية)

لماذا MongoDB؟

- (Seller → Product → Sale) علاقات واضحة ومحكمة بين الجداول
- بيع منتج + نقصان مخزون لازم يصيروا) قوية Transactions (atomic – سوا أو ما يصير ولا واحد فيهم)
- اللازمة aggregations (SUM, GROUP BY) قوي بالـ للتقارير
- Prisma + TypeScript يتكامل منيح مع

8. Multi-Tenancy (كيف بنفصل بيانات كل Account)

بس كل جدول فيه `Shared database, shared schema`، `accountId`.

- seller منفصلة لكل database مش (overkill 10 JK accounts)
- `WHERE accountId = ...` بتتفلتر إجبارياً بـ query كل حتى ما ينسى حدا backend بالـ middleware عن طريق –
(تاني seller يشوف بيانات seller: خطير لو نسي bug) يفلتر

9. Database Schema (Prisma) – النسخة النهائية

```
enum UserRole {
  SUPER_ADMIN
  OWNER
  WAREHOUSE
}

model User {
  id      String  @id @default(cuid())
  email   String  @unique
  password String
  role    UserRole
  accountId String? // فقط لـ null
  SUPER_ADMIN
  account Account? @relation(fields:
```

```

[accountId], references: [id])
  createdAt DateTime @default(now())
}

model Account {
  id          String      @id
  @default(cuid())
  name        String
  isActive    Boolean     @default(true)
  users       User[]
  categories  Category[]
  products    Product[]
  expenses    Expense[]
  sales       Sale[]
  createdAt   DateTime    @default(now())
}

model Category {
  id          String      @id
  @default(cuid())
  accountId   String
  account     Account     @relation(fields:
[accountId], references: [id])
  name        String
  profitMargin Float      @default(0.3)
  products    Product[]
  createdAt   DateTime    @default(now())

  @@index([accountId])
}

model Product {
  id          String      @id
  @default(cuid())

```

```

    accountId      String
    account        Account
    @relation(fields: [accountId], references:
    [id])
    categoryId      String
    category        Category
    @relation(fields: [categoryId], references:
    [id])
    name            String
    imageUrl        String
    color           String
    wholesaleCost   Float?          // null لحد
    يحطها Owner ما ال
    currentStock    Int
    @default(0)
    suggestedPrice   Float?          // null لحد
    cost ما يصير عندها
    createdById     String
    createdAt       DateTime
    @default(now())
    sales            Sale[]
    movements        StockMovement[]

    @@index([accountId])
    @@index([accountId, categoryId])
}

model StockMovement {
  id          String  @id @default(cuid())
  productId   String
  product     Product @relation(fields:
[productId], references: [id])
  quantity    Int
  costAtTime  Float?

```

```
    addedById String
    createdAt DateTime @default(now())

    @@index([productId])
}

model Expense {
  id          String    @id @default(cuid())
  accountId   String
  account     Account   @relation(fields:
[accountId], references: [id])
  description String
  amount      Float
  date        DateTime @default(now())

  @@index([accountId])
}

model Sale {
  id          String    @id @default(cuid())
  accountId   String
  account     Account   @relation(fields:
[accountId], references: [id])
  productId   String
  product     Product   @relation(fields:
[productId], references: [id])
  quantity    Int
  soldPrice   Float
  soldById    String
  createdAt   DateTime @default(now())

  @@index([accountId, createdAt])
}
```

```
@@index([productId])
}
```

10. اعتبارات الأداء (Scale)

هل 10,000 مستخدم مشكلة؟

- بنفس **requests** مسجلين $\neq$ 10,000 users 10,000
الثانية – الفرق جوهري
- (عام بترافيك عشوائي app مش) بما إنه نظام داخلي
الاستخدام مؤزّع عبر ساعات الدوام، مش كله بلحظة وحدة
- مش آلاف peak، بال RPS التقدير الواقعي: عشرات إلى مية

مشكلة (منتج 500 × 10K account) هل 5 مليون منتج

Postgres/Supabase؟

- بيتحمل بسهولة عشرات وحتى مئات Postgres – لا إطلاقاً
الملايين من الصفوف
- موجود أعلاه بال) صحيح **Indexing**: الشرط الوحيد
بترجع ليستة query بكل **pagination** + (schema)
- البيانات النصية/الرقمية (بدون صور): GB من ناحية الحجم بال
مدفوع plan أقل بكثير من حدود أي GB – رح تضل تحت 1-2
بـ Supabase

Premature مبدأ عام تم اعتماده: تجنب الـ
Optimization

- متعددة) ما نبني من اليوم الأول لسيناريوهات ضخمة نظرية (إلخ، Redis، السيرفرات
 - (server + Postgres + connection pooling) نبلس ببنية بسيطة تشتغل صح (بسيط
 - بس لما (caching، horizontal scaling) نضيف تحسينات (تصير مشكلة أداء حقيقية وملموسة
-

11. الخطوات القادمة (Next Steps)

1. تجهيز الـ project structure (backend + frontend)
2. migration فعلياً وعمل أول Prisma schema تطبيق الـ
3. Auth + Roles بناء نظام الـ
4. (Warehouse flow) إضافة منتج: feature بناء أول
5. بناء منطق حساب السعر الأوتوماتيكي
6. بناء صفحة تسجيل البيع ونقصان المخزون
7. بناء الجرد اليومية/الأسبوعية